

Reviewer Pack — Tropicalized Belyi Functions and Resolution Proof Depth for ...

Entry 03ec2444916e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 03:58:56 UTC

Tropicalized Belyi Functions and Resolution Proof Depth for Tseitin Formulas

Entry ID: 03ec2444916e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 03:58:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Tropical Geometry) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

[‘For every Tseitin formula F over a graph G , there exists a tropicalized Belyi function φ associated with G such that the resolution proof depth of F is at least $2^{(\log(|G|) * \Omega(\nu(G)))}$, ‘where $\nu(G)$ is a poly-time computable invariant of G and $|G|$ denotes the number of vertices in G .’, ‘Furthermore, for non-expanders, $\nu(G) = O(1)$.’]

Rationale (proposer’s reasoning):

[‘Tropicalized Belyi functions are known to relate algebraic curves and their properties to computational complexity (e.g., as seen in tropicalization techniques).’, ‘The resolution proof depth of Tseitin formulas is a key measure for proving lower bounds in complexity theory, especially for NP-complete problems.’, ‘A direct link between these two fields could potentially reveal new insights into the structure of Tseitin formulas and their proof complexity.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b96faff226754019

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on tropicalized Belyi functions and resolution proof depth for Tseitin formulas, support is indicated if, across 30 random graph seeds, the correlation coefficient between $\nu(G)$ and the resolution proof depth of F exceeds 0.8 with a p-value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropicalized Belyi functions' AND 'resolution proof depth' AND 'Tseitin formulas' - 'algebraic geometry' AND 'complexity theory' AND 'resolution proof complexity' - 'tropical geometry' AND 'resolution proof depth' AND 'graph G'

Top relevant hits considered: - [http://arxiv.org/abs/1712.05268v1] Belyi map for the sporadic group J2 - [http://arxiv.org/abs/1704.06419v1] Belyi map for the sporadic group J1 - [http://arxiv.org/abs/1703.05200v1] The sporadic group J2, Hauptmodul and Belyi map - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity - [http://arxiv.org/abs/2211.07055v3] Geometric complexity theory for product-plus-power - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmüller and Siegel spaces - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        edges = set()
        for _ in range(n * (n - 1) // 2):
            u, v = random.sample(range(n), 2)
            if u < v and (u, v) not in edges and (v, u) not in edges:
                edges.add((u, v))
        return list(edges)

    def tseitin_formula(graph):
        n = len(graph)
        literals = {i: f'x{i}' for i in range(n)}
        clauses = []

        # Add clauses for each edge
```

```

    for u, v in graph:
        clauses.append([literals[u], literals[v]])
        clauses.append([-literals[u], -literals[v]])

    # Add clauses for each vertex
    for i in range(n):
        clauses.append([literals[i]])
        for j in range(i + 1, n):
            clauses.append([-literals[i], -literals[j]])

    return clauses

def resolution_proof_depth(clauses):
    stack = []
    while True:
        new_clause = None
        for clause in clauses:
            if len(clause) == 1:
                literal = clause[0]
                if literal.startswith('-'):
                    negated_literal = literal[1:]
                    if negated_literal in [c for c in stack]:
                        stack.remove(negated_literal)
                else:
                    new_clause = [-negated_literal]
                    break
            elif literal not in stack:
                stack.append(literal)
            elif len(clause) == 2 and clause[0].startswith('-') and clause[1] == -clause[0][1:]:
                stack.remove(-clause[0][1:])
        if new_clause is None:
            return len(stack)

def poly_time_invariant(graph):
    n = len(graph)
    # Example invariant: number of edges
    return n * (n - 1) // 2

n = random.choice([5, 10, 15, 20, 30, 40])
graph = generate_graph(n)
clauses = tseitin_formula(graph)
depth = resolution_proof_depth(clauses)
invariant = poly_time_invariant(graph)

return {
    "metric_name": "Resolution Proof Depth",
    "metric_value": depth,
    "instances_tested": 1,
    "conjecture_holds": depth >= 2 ** (math.log(n) * math.log(invariant)),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_depth = sum(r["metric_value"] for r in results) / len(results)
std_depth = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_985b1c8f.py", line 93, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_985b1c8f.py", line 75, in run_trial
    clauses = tseitin_formula(graph)
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_985b1c8f.py", line 37, in tseitin_formula
    clauses.append([-literals[u], -literals[v]])
                   ~~~~~^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and p-value as required by the support condition. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14845
2	preregistration	ollama_remote	glm4:latest	0	0	10644
3	novelty	ollama_remote	glm4:latest	0	0	8766
4	novelty	ollama_remote	glm4:latest	0	0	9529
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14484
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9173
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7806
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10153
9	judge	ollama_remote	glm4:latest	0	0	11333

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96733 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/03ec2444916e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/03ec2444916e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/03ec2444916e.tar.gz` (if generated)